

```
rpm -Uvh http://download.fedoraproject.org/pub/epel/5/i386/epel-release-5-3.noarch.rpm
```

3. Install **puppet** and **ruby-ldoc**:

```
yum install puppet ruby-ldoc
```

This completes installation of the Puppet server and client.

Before proceeding further, make sure that the systems timing for the Puppet server and client are in sync. Now, from the client, issue the following command to get approval from the server as its subscriber:

```
puppetd -verbose -server puppet.domain.com
```

This will display the following output:

```
info: Creating a new certificate request for pclient.torridnetworks.com
info: Creating a new SSL key at /var/lib/puppet/ssl/private_keys/
puppetclient.domain.com.pem
```

In the above command, the client has raised a request to the server to be registered as a subscriber. Now, the server needs to approve the subscriptions. To view the pending subscriptions, issue the following command on the server:

```
puppetca -list
```

The above command will give the name of the node that needs to be approved or signed by the server. In the next command, sign that node:

```
puppetca -s puppetclient.domain.com
```

Once the client is approved by the server, the class assigned to the client will be executed. In this case, a file `/tmp/testfile` will be created on `puppetclient.domain.com`. If the created file is deleted, it will be recreated on the next polling, i.e., within the next 30 minutes.

Once the basic Puppet infrastructure is ready, different classes can be created to accomplish different tasks.

Some sample Puppet classes

Below are a few sample classes for quick reference.

Sample 1: To install Apache and run the `httpd` service:

```
class apache {
  package { ['httpd'] ensure => installed }
  service { ['httpd']
    ensure => running,
    require => Package['httpd'],
  }
}
```

Sample 2: To stop the `mdm` service:

```
class redhat {
  service {
    'mdm':
      enable => true,
      ensure => stopped,
  }
}
```

Sample 3: To execute commands:

```
class start_vhost {
  $noop = true
  exec { ['/usr/sbin/start_ws',]
  exec { ['/usr/sbin/start_vhost',]
```

Sample 4: To start a service as per the remote operating system:

```
class httpd_service_start {
  case $operatingsystem {
    redhat { service { ['httpd'] ensure => running } }
    debian { service { ['apache'] ensure => running } }
    default { service { ['apache2'] ensure => running } }
  }
}
```

Sample 5: To create a user:

```
class virt_users {
  [user { ['jsmith']:
    ensure => 'present',
    uid => '507',
    gid => '507',
    comment => 'John Smith',
    home => '/nfs/HR/home/jsmith',
    shell => '/bin/bash',
  }
}
```

Sample 6: To manage Cron job:

```
class set_cron_syscheck {
  cron { ['syscheck']:
    command => '/usr/bin/syscheck',
    user => 'root',
    hour => '18',
    minute => '0'
  }
}
```

Sample 7: Transferring a file from the Puppet server:

```
class httpd_conf {
  file { ['httpd.conf']:
    source => 'puppet://puppetmaster/httpd/conf/httpd.conf'
  }
}
```

Of course, much more detailed manifests can be created to manage multiple servers with heterogeneous UNIX operating systems. Subversion can be configured with Puppet to store configuration files and track changes, so that the changes can be reverted to a previous state.

Reporting is one of the important aspects of a configuration management system. Reporting from a configuration management system can provide information on performance and compliance to policies and standards. Puppet's reporting engine is limited at this stage, but still allows some useful basic reporting that can be graphed and displayed.

So, all in all, Puppet can be a real boost for UNIX administrators. **END** 

By: Dhruv Soli

The author is the founder and principal consultant at Torrid Networks Pvt Ltd, and chairs OWASP India. Torrid specialises in information security and open source consulting services. OWASP is a worldwide free and open community focused on improving the security of application software. Dhruv can be reached at dhruv.soli@torridnetworks.com